

GDS

Dataset movie recommendations

Etapes à suivre

1. Stop votre instance sorbonne
2. Copier le fichier recommendations.dum dans le dossier import
3. Ouvrir une terminal à partir du dossier bin de votre instance
4. Executer la commande suivante en donnant le chemin complet du dossier import

```
.\neo4j-admin.bat database load recommendations --from-path="path complete du dossier import" --overwrite-destination=true
```

Exemple :

```
.\neo4j-admin.bat database load recommendations
  --from-path="C:\Users\33641\Neo4jDesktop2\Data\dbms\dbms-e800f016-787d-41ee-bc75-dba76f3265dd\import"
  --overwrite-destination=true
```

1. Start votre instance sorbonne
2. Se connecter sur la base recommendations .

Executer la requete : Lister les projections existantes

```
CALL gds.graph.list()
```

.

Executer la requete : Creation d'une projection

```
CALL gds.graph.project(
  'my-graph-projection',
  ['Actor', 'Movie'],
  'ACTED_IN'
)
```

.

Executer la requete : Lister les projections existantes

```
CALL gds.graph.list()
```

Resultat

```
1 CALL gds.graph.project(
2   'my-graph-projection',
3   ['Actor','Movie'],
4   'ACTED_IN'
5 )
```

nodeProjection	relationshipProjection	graphName	nodeCount	relationshipCount	projectMillis
{ Actor: { label: "Actor", properties: {} }, Movie: { label: "Movie", properties: {} } }	{ ACTED_IN: { aggregation: "DEFAULT", orientation: "NATURAL", indexInverse: false, properties: {}, type: "ACTED_IN" } }	"my-graph-projection"	24568	35910	340

. Executer la requete

```
CALL gds.graph.list()
YIELD graphName, nodeCount, relationshipCount, schema
```

Resultat

```
1 CALL gds.graph.list()
2 YIELD graphName, nodeCount, relationshipCount, schema
```

graphName	nodeCount	relationshipCount	schema
"my-graph-projection"	24568	35910	{ graphProperties: {}, nodes: { Actor: {}, Movie: {} }, relationships: { ACTED_IN: {} } }

1. Degree centrality sur les nodes Actor

```
CALL gds.degree.mutate('my-graph-projection',
  {mutateProperty: 'numberOfMoviesActedIn'})
```

Resultat

recommendations\$ CALL gds.degree.mutate('my-graph-projection', {mutateProperty: 'numberOfMoviesActedIn'})

nodePropertiesWr	preProcessingMill	computeMillis	postProcessingMil	mutateMillis	centralityDistribution	configuration
1 24568	8	6	41	9	{ min: 0.0, max: 56.00024414062499, p90: 3.0000076293945312, p999: 33.00023651123047, p99: 16.00011444091797, p50: 1.0, p75: 1.0, p95: 6.000022888183594, mean: 1.4616605882262688 }	{ orientation: "NATURA mutateProperty: "num iesActedIn", jobId: "790d24fc-466 611-5693f3096928", logProgress: true, nodeLabels: ["*"], relationshipTypes: [concurrency: 4, sudo: false }

1. Streaming

```
CALL gds.graph.nodeProperty.stream(
  'my-graph-projection',
  'numberOfMoviesActedIn'
)
YIELD nodeId, propertyValue
RETURN
  gds.util.asNode(nodeId).name AS actorName,
  propertyValue AS numberOfMoviesActedIn
ORDER BY numberOfMoviesActedIn DESCENDING, actorName LIMIT 10
```

Resultat

```
1 CALL gds.graph.nodeProperty.stream(
2   'my-graph-projection',
3   'numberOfMoviesActedIn'
4 )
5 YIELD nodeId, propertyValue
6 RETURN
7   gds.util.asNode(nodeId).name AS actorName,
8   propertyValue AS numberOfMoviesActedIn
9 ORDER BY numberOfMoviesActedIn DESCENDING, actorName LIMIT 10
```

actorName	numberOfMoviesA
3 "Nicolas Cage"	45.0
4 "Samuel L. Jackson"	45.0
5 "Clint Eastwood"	40.0
6 "Michael Caine"	40.0
7 "Gene Hackman"	38.0
8 "John Cusack"	38.0
9 "John Travolta"	38.0

1. Ecriture

```
CALL gds.graph.nodeProperties.write(
  'my-graph-projection',
  ['numberOfMoviesActedIn'],
  ['Actor']
)
```

)

Resultat

```
1 CALL gds.graph.nodeProperties.write(  
2   'my-graph-projection',  
3   ['numberOfMoviesActedIn'],  
4   ['Actor']  
5 )
```

Table RAW

writeMillis	graphName	nodeProperties	propertiesWritten	configuration
459	"my-graph-projection"	["numberOfMoviesActedIn"]	15443	{ jobId: "df5ba717-de5e-44b6-95c7-c95c2e662ae8", logProgress: true, concurrency: 4, sudo: false, writeToResultStore: false, writeConcurrency: 4 }

```
MATCH (a:Actor)  
RETURN a.name, a.numberOfMoviesActedIn  
ORDER BY a.numberOfMoviesActedIn DESCENDING, a.name LIMIT 10
```

Resultat

File Edit View Window Help Developer

```
1 MATCH (a:Actor)  
2 RETURN a.name, a.numberOfMoviesActedIn  
3 ORDER BY a.numberOfMoviesActedIn DESCENDING, a.name LIMIT 10
```

Table RAW

a.name	a.numberOfMoviesActedIn
"Robert De Niro"	56.0
"Bruce Willis"	49.0
"Nicolas Cage"	45.0
"Samuel L. Jackson"	45.0
"Clint Eastwood"	40.0
"Michael Caine"	40.0
"Gene Hackman"	38.0
"John Cusack"	38.0
"John Travolta"	38.0
"Morgan Freeman"	38.0

GDS Native projection

La projection native permet de gérer la direction des Relationships Pour cela :

NATURAL: Meme direction que ce qui est dans database (default) REVERSE: direction opposée
UNDIRECTED: undirected

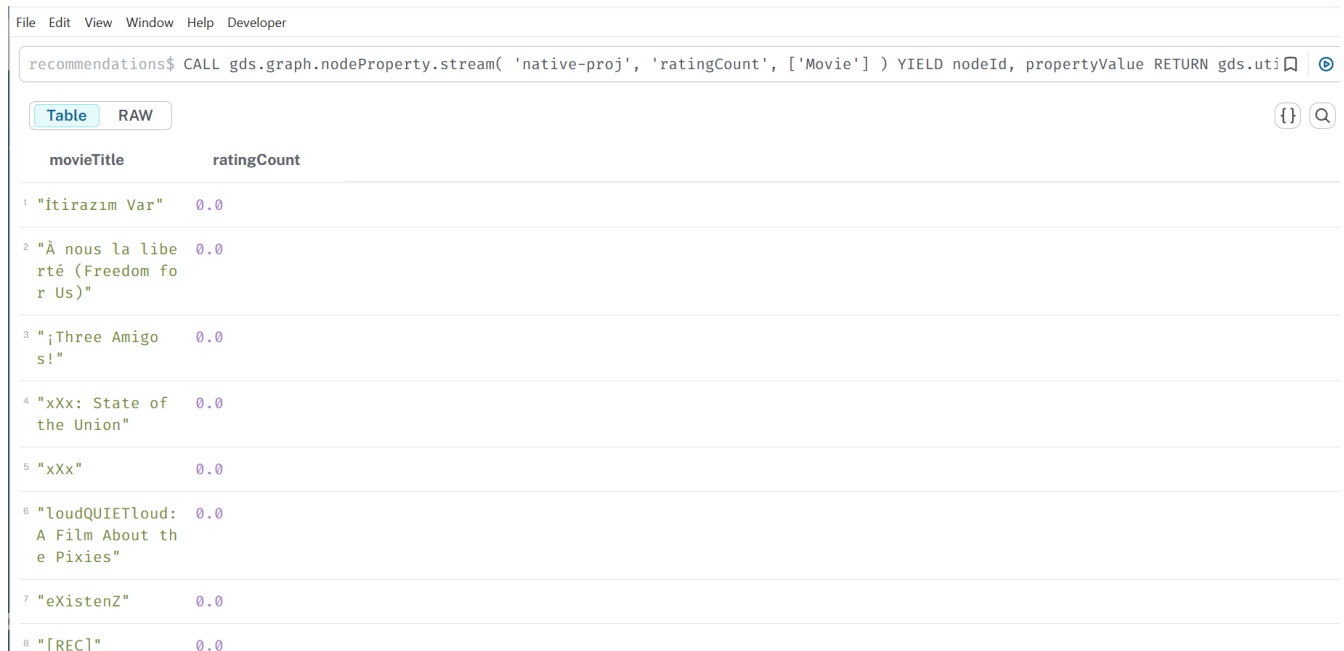
Prenons l'exemple du Graph que nous venons de projeter. Imaginons que nous souhaitions compter le nombre d'évaluations attribuées à chaque film par les utilisateurs. Si nous utilisons

l'appel de degré comme lors de la dernière leçon, nous obtiendrons uniquement des zéros.

```
CALL gds.graph.drop('native-proj', false);
CALL gds.graph.project(
  'native-proj',
  ['User', 'Movie'],
  ['RATED']
);
CALL gds.degree.mutate(
  'native-proj',
  {mutateProperty: 'ratingCount'}
);
```

Resultat :

```
CALL gds.graph.nodeProperty.stream(
  'native-proj',
  'ratingCount',
  ['Movie']
)
YIELD nodeId, propertyValue
RETURN
  gds.util.asNode(nodeId).title AS movieTitle,
  propertyValue AS ratingCount
ORDER BY movieTitle DESCENDING LIMIT 10
```



The screenshot shows a Neo4j Cypher console interface. At the top, there's a menu bar with 'File', 'Edit', 'View', 'Window', 'Help', and 'Developer'. Below it is a search bar containing the query: `recommendations$ CALL gds.graph.nodeProperty.stream( 'native-proj', 'ratingCount', ['Movie'] ) YIELD nodeId, propertyValue RETURN gds.util.asNode(nodeId).title AS movieTitle, propertyValue AS ratingCount ORDER BY movieTitle DESCENDING LIMIT 10`. Below the search bar, there are two tabs: 'Table' (selected) and 'RAW'. To the right of the tabs are icons for a code editor, a search icon, and a refresh icon. The main area displays a table with two columns: 'movieTitle' and 'ratingCount'. The table contains 8 rows of data, each with a line number in the left margin.

	movieTitle	ratingCount
1	"İtirazım Var"	0.0
2	"À nous la liberté (Freedom for Us)"	0.0
3	"¡Three Amigos!"	0.0
4	"xXx: State of the Union"	0.0
5	"xXx"	0.0
6	"loudQUIETloud: A Film About the Pixies"	0.0
7	"eXistenZ"	0.0
8	"[REC]"	0.0

Ce résultat est lié à la direction de la Relationship

Pour Corriger ce problème :

```
CALL gds.graph.drop('native-proj', false);
```

```
//replace with a project that has reversed relationship orientation
CALL gds.graph.project(
  'native-proj',
  ['User', 'Movie'],
  {RATED_BY: {type: 'RATED', orientation: 'REVERSE'}}
);

CALL gds.degree.mutate(
  'native-proj',
  {mutateProperty: 'ratingCount'}
);
```

Resultat :

```
CALL gds.graph.nodeProperty.stream(
  'native-proj',
  'ratingCount',
  ['Movie']
)
YIELD nodeId, propertyValue
RETURN
  gds.util.asNode(nodeId).title AS movieTitle,
  propertyValue AS ratingCount
ORDER BY movieTitle DESCENDING LIMIT 5
```

File Edit View Window Help Developer

```
1 CALL gds.graph.nodeProperty.stream(
2   'native-proj',
3   'ratingCount',
4   ['Movie']
5 )
6 YIELD nodeId, propertyValue
7 RETURN
8   gds.util.asNode(nodeId).title AS movieTitle,
9   propertyValue AS ratingCount
10 ORDER BY movieTitle DESCENDING LIMIT 5
```

Table RAW

	movieTitle	ratingCount
1	"İtirazım Var"	1.0
2	"À nous la liberté (Freedom for Us)"	1.0
3	"¡Three Amigos!"	31.0
4	"xXx: State of the Union"	1.0
5	"xXx"	23.0

GDS Cypher projection

Dans la partie Native projection , nous avons déterminé quels acteurs étaient les plus prolifiques en fonction du nombre de films dans lesquels ils ont joué. Supposons plutôt que nous souhaitions savoir quels acteurs sont les plus influents en termes de nombre d'autres acteurs avec lesquels ils

ont joué dans des films récents à gros budget.

Pour cet exemple, nous qualifierons un film de «RECENT» s'il est sorti en 1990 ou après, et de gros budget s'il a généré des recettes supérieures à 1million de dollars.

Le Graph n'est pas configuré pour répondre correctement à cette question avec une projection native directe. Cependant, nous pouvons utiliser une projection chiffrée pour filtrer les nœuds appropriés et effectuer une agrégation afin de créer une relation ACTED_WITH dont la propriété actedWithCount relie directement les nœuds acteurs.

1. Requete cypher

```
// requete cypher
MATCH (source:Actor)-[r:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-(target)
WHERE m.year >= 1990 AND m.revenue >= 1000000
RETURN source.name, count(r) as actedWithCount
```

1. Creation de la projection

```
// Creation de la projection
MATCH (source:Actor)-[r:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-(target)
WHERE m.year >= 1990 AND m.revenue >= 1000000
WITH source, target, count(r) as actedWithCount
WITH gds.graph.project(
    'cypher-proj',
    source,
    target,
    { relationshipProperties:
        {
            actedWithCount: actedWithCount
        }
    }
) AS g
RETURN
    g.graphName AS graph, g.nodeCount AS nodes, g.relationshipCount AS rels
```

1. Application de Degree centrality

```
// Degree centrality
CALL gds.degree.stream('cypher-proj',{relationshipWeightProperty:
'actedWithCount'})
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS name, score
ORDER BY score DESC LIMIT 10
```

Resultat :

```
1 CALL gds.degree.stream('cypher-proj',{relationshipWeightProperty: 'actedWithCount'})
2 YIELD nodeId, score
3 RETURN gds.util.asNode(nodeId).name AS name, score
4 ORDER BY score DESC LIMIT 10
```

Table RAW

	name	score
1	"Robert De Niro"	123.0
2	"Bruce Willis"	120.0
3	"Johnny Depp"	102.0
4	"Denzel Washington"	99.0
5	"Nicolas Cage"	90.0
6	"Brad Pitt"	87.0
7	"Julianne Moore"	87.0
8	"Samuel L. Jackson"	85.0